

מבוא לתקשורת מחשבים הרצאה 4

הגדרה: רשת מקומית

כולם מתחברים אליה ויש לו יכולת של broadcast. למשל ethernet, wifi וכו'.

כמה גדולה יכולה להיות הרשת המקומית?

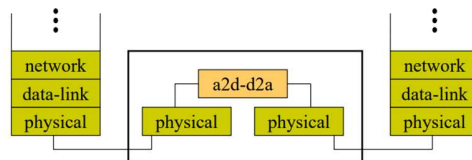
צריך לקחת בחשבון:

- נכונות – בעיה פיזיקלית, טווח האותות מוגבל. לכן אם נשדר הודעה ארוכה מדי זה ידפוק את הנכונות.
- ביצועים – אם התווח משותף לכולם אז מידת התקשורת קבועה אז ככל שנחבר יותר תחנות יהיה להן בממוצע פחות יכולת לשדר.

פתרונות:

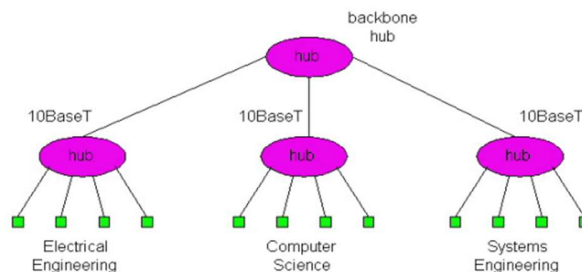
Repeater

הבעיה היא שהסיגנל נחלש לאורך הדרך, לכן יש מכשיר פשוט שמתקן את זה שנקרא repeater. הוא מנקה את הרעש ושולח את האות מחדש בעזרת תרגום מאנלוגי לדיגיטלי ובחזרה.



Hub

Hub זה repeater עם הרבה כניסות ויציאות. הוא קולט אות ומשדר אותו לכל הרגליים, כלומר הוא גם משכפל את האות. הhub צריך אספקת כוח הוא אינו רכיב פסיבי. אם הוא מזהה שידור 2 תחנות הוא ישדר רעש וזה יסמל התנגשות. כל התחנות מחוברות לhub.

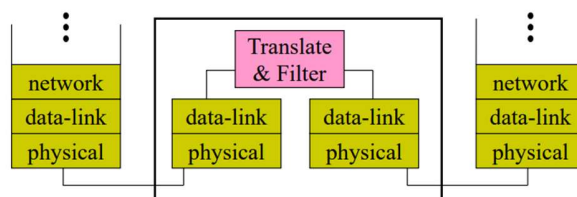


בעיה: הוא הופך את כולם להיות מחוברים כיאילו לאותו תווח. לכן כולם ישמעו את השידור ונקבל collision domain משותף.

פתרון: bridge

Bridge / Ethernet Switch

דומה לhub אבל בתוכו יש מימוש של data link, כלומר הוא בשכבה 2. הגשר יעשה עבודה יותר אינטלגנטית מאשר רק לנקות את הסיגנל ולשכפל אותו. הוא יכול לחבר data link ימים מסוגים שונים (ethernet, wifi, token ring) והכי חשוב: הוא לא מקדם כל דבר בלי לחשוב. קוראים לזה לפעמים גשר לומד, הגשר יאזין לתעבורה, ממנה הוא ילמד איפה נמצאות התחנות ויתחזק טבלה שבה הוא רושם באיזה רגל הוא חושב שנמצאת כל תחנה. לפעמים קוראים לו גם switch אבל אנחנו נשמור את המושג הזה לשכבה 3.

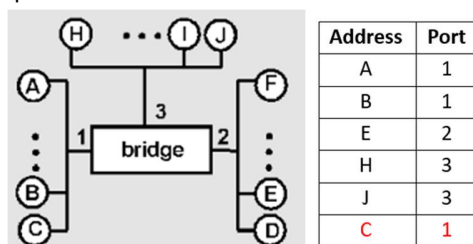


אלגוריתם

- כשהגשר שומע הודעה, הוא יודע מאיזה רגל p הוא שמע ומהי כתובת השולח SA ומהי כתובת היעד DA.
- הוא מכניס את (SA, p) לטבלה (בלי היעד).
- אם היעד הוא broadcast או שהיעד לא נמצא בטבלאות אז משדרים לכולם.
- אם היעד כן נמצא בטבלה, אז הוא משדר את ההודעה רק לרגל q.
- לכל שורה בטבלה יש זמן חיים מוגבל (כמה דקות). אם אחרי כמה דקות לא שמענו הודעה מאותו יעד השורה תמחק. זה חשוב כי אם אנחנו מעבירים מחשב ממקום אחד למקום אחר צריך לעדכן את הטבלאות.

דוגמא

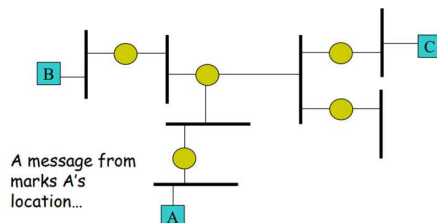
לגשר יש 3 רגליים והתחנות מסומנות באותיות. נניח שהטבלה כרגע זה מה שמופיע בצד ימין.



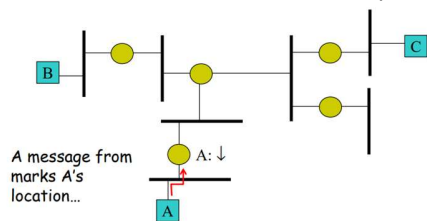
- נניח ש-C שולח הודעה ל-D.
- D לא נמצא בתוך הטבלה, הגשר רשם ש-C יושב על רגל מס' 1 ואז שלח את ההודעה לרגליים 2 ו-3.
- D עונה ל-C והגשר כבר יודע איפה C נמצא בגלל ההודעה הראשונה שהוא שלח.

דוגמא

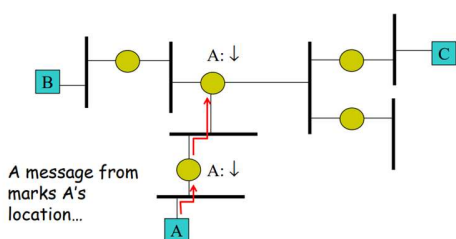
כל קו עבה זה רשת, הקווים הדקים הם רגליים, העיגולים זה גשרים והריבועים הם תחנות. נניח שאנחנו משדרים הודעה מ-A ושהזיכרון של כל הגשרים ריק.



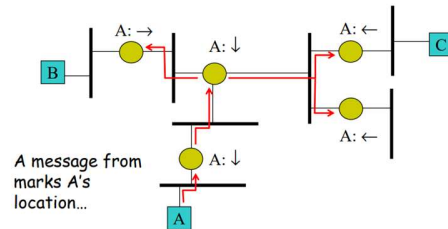
ההודעה מתפשטת למטה והגשר רושם את המיקום של A ואז שולחים את ההודעה לכל הרגליים האחרות.



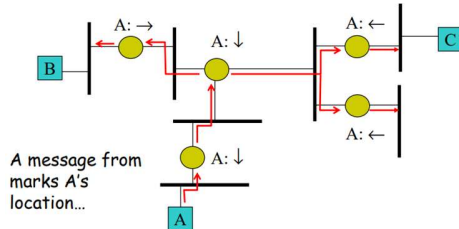
כעת ההודעה מגיעה גם לגשר הזה וגם הוא רושם את המיקום של A.



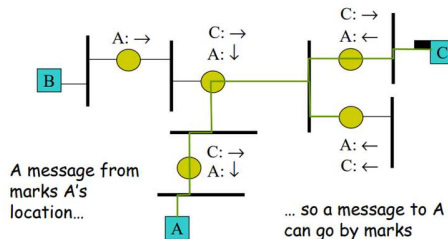
ההודעה מתפשטת ימינה ושמאלה.



ובסוף זה מגיע גם ל-C (וגם לכולם). כעת כל הגשרים יודעים איפה נמצא A.

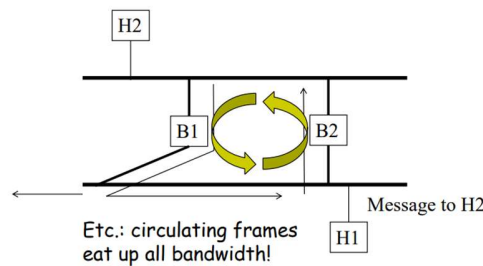


כעת C רוצה לענות ל-A והוא שולח לו הודעה וההודעה יודעת לאן ללכת במסלול האופטימלי.



בעיה: לולאות

נניח שיש לנו 2 גשרים שמחברים את אותם שני סגמנטים ו-H1 שולח הודעה. B1 ו-B2 זוכרים ש-H1 למטה. ההודעה עוברת על הענף העליון וכעת B1 יחשוב ש-H1 נמצא על הרגל העליונה. ואז B1 ישדר אותה שוב על הענף התחתון. מתקבל מעגל אינסופי.



פתרון: spanning tree protocol

Spanning tree protocol פרוטוקול העץ הפורש

זה לא הגיוני שבאשר מוסיפים משאבים הביצועים מחמירים. לכן הפתרון הוא שחלק מהכבלים לא יהיו בשימוש (יהיו עדיין מחוברים) והחלק של הגשרים שכן ישתמשו בו יהיה קשיר וחסר מעגלים. הגשרים מריצים את הפרוטוקול באופן מבוזז. כל גשר מריץ תכנית, הם מדברים אחד עם השני ובסוף הם מחליטים איזה פורטים הם משביתים. הפרוטוקול יהיה בשכבות: כל שכבה תפתור בעיה אחת.

שליבים

1. בחירת גשר שורש

לכל גשר יש ID משלו. כל פעם כל אחד מפיץ את ה-ID הכי קטן שהוא שמע עליו. השורש יהיה הגשר עם ה-ID הנמוך ביותר. בעיה: אם הלך הספק או השקע נדפק והגשר עם ה-ID הנמוך נדפק, אין להם דרך לדעת זאת. הם חושבים שהוא השורש אבל הוא מת.

2. בהינתן שורש, נבנה את העץ של הגשרים

נבנה עץ עם מרחקים קצרים (עץ BFS). לשם כך יש אלגוריתם Bellman-Ford.

Bellman-Ford

האלגוריתם דורש שורש source. כל צומת מדי פעם מספרת לשכניו מה הערכת המרחק שלה מ-s, נסמן אותו בdist. עבור השורש המרחק הוא $dist_s = 0$. עבור שאר הצמתים u, מחשבים את המרחק שלנו ע"י הוספת 1 למרחק של השכן u:

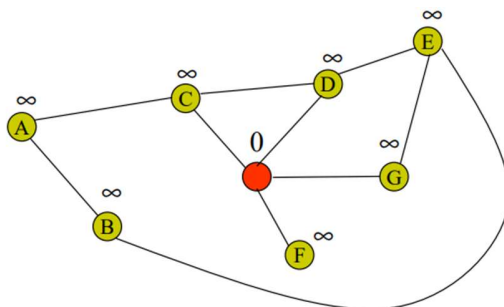
$$cost_v(u) := dist_u + 1$$

הצומת בסוף בוחרת את המרחק הקטן ביותר:

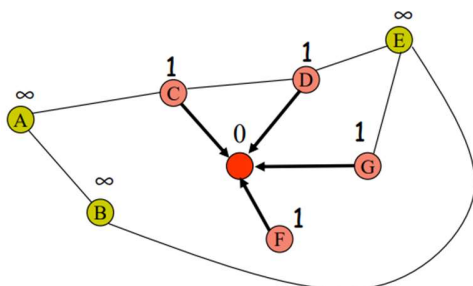
$$dist_v := \min\{cost_v(u) \mid u \text{ is a neighbor of } v\}$$

זה אלגוריתם שרץ כל הזמן ובאיזה שהוא שלב הוא מתייצב.

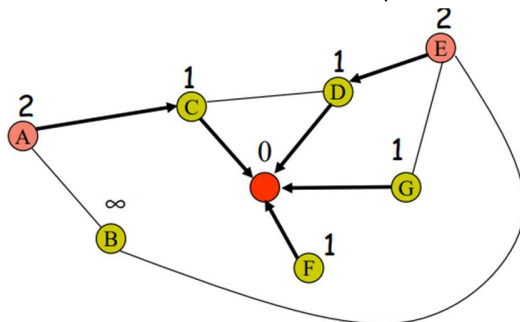
דוגמא:



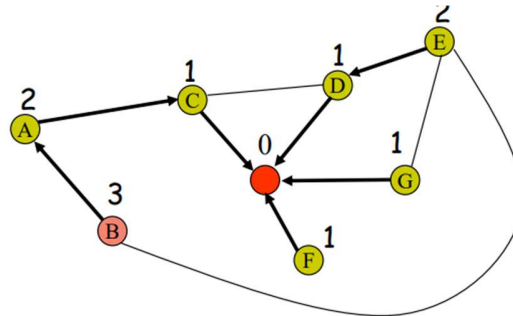
השורש מסומן באדום ומרחקו הוא 0. כל הצמתים האחרים מרחקים אינסוף. נניח לפשטות שהמערכת מתקדמת באופן מסונכרן, בכל סיבוב כל אחד שולח את ההודעות, מקבל את ההודעות ומעדכנים את המשתנים. אחרי סיבוב אחד כל השכנים של השורש שמעו על השורש, מסמנים את מרחקם כ-1 ומסמנים את הרגל של השורש.



בעת השכנים של השכנים שמעו ויעדכו את מרחקם:



בסיבוב הבא B ישמע גם מ A וגם מ E ולכן הוא יבחר את הרגל שמצביעה ל ID הנמוך יותר.



תכונות של האלגוריתם:

- אפשר לעדכן המרחקים של הקשתות להיות שונים מ-1:

$$cost_v(u) := dist_u + W(u, v)$$
- המערכת תעבוד גם תחת תזמון לא סינכרוני (לא נראה זאת). זאת תכונה קריטית כי קשה לסנכרן את כל הרשת ולכן נרצה שהאלגוריתם יעבוד גם כשכל גשר עובד בקצב שלו.
- המערכת תתייצב גם אם לא כתוב בהתחלה אינסוף, כלומר אין קשר למה שהיה בעבר.

3. בניית עץ פורש של הסגמנטים

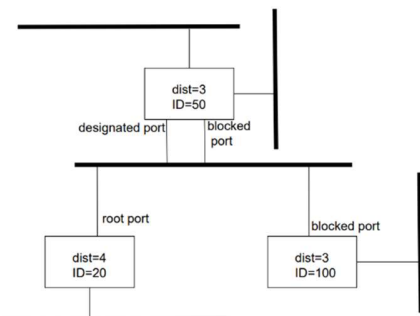
בהינתן עץ פורש של הגשרים נרצה לבנות כעת עץ פורש של הסגמנטים.

לכל סגמנט יהיה רגל אחת בדיוק אשר מובילה לרגל השורש ונקרא לה **designated port** (הרגל הנבחרת).

אם יש להם אותו מרחק, נבחר את הפורט עם ה-ID הכי קטן. בכל רגל הגשרים מאזינים להודעות של הרגליים. הגשר מסמן את הרגל בתור הרגל הנבחרת אם היא הכי טובה לרגל הזאת – כלומר אם ההודעה שהוא משדר יותר טובה מכל ההודעות שהוא מקבל.

דוגמא

יש כאן 2 רגליים שמחוברות לאותו גשר. אם נסתכל על הסגמנט האמצעי, אפשר להסיק שהשורש הוא הרגל השמאלית למטה. למעלה בשמאל יש את המרחק הכי קטן ו-ID יותר קטן ולכן נבחר אותה כרגל הנבחרת. ובסוף האלגוריתם יחסום פורטים שהם לא שורש ולא נבחרים. כלומר הם לא משתתפים בהעברת הודעות.



מימוש האלגוריתם STP

כל אחד שולח את השם של השורש לפי דעתו, את המרחק לפי דעתו ואת ה-ID שלו, זה נקרא (BPDU) Bridge Protocol Data Unit.

(myRoot, myDist, myID)

מסתכלים על לקסיקוגרפית – משווים את השדות הראשונים. אם הם זהים, משווים את השדות השניים. אם הם זהים, משווים את ה-ID. תמיד יש הודעת ברירת מחדל שאומרת אני השורש של עצמי, המרחק מהשורש הוא 0 וזה השם שלי:

(myID, 0, myID)

וכמו שאמרנו, הוא שוכח הודעות ישנות אחרי כמה זמן.

יש לנו רשת כלשהי, אנחנו יודעים שהיא קשירה, אנחנו משתמשים בה כדי לייצר STP ועל כל הרגליים משדרים את ה-BPDU-ים. חלק מהפורטים יוגדרו root, blocked, designated.

יש כאן 2 מישורים:

- מישור הבקרה שמייצר את העץ.
- מישור הדאטא שבו ההודעות מתקדמות רק על העץ.